

PFO 2: Sistema de Gestión de Tareas con API y Base de Datos

Documentación, pruebas y respuestas conceptuales

Materia: Programación sobre redes

Alumno: Adriano Caloni - Comisión E

Repositorio: <https://github.com/AdrianoJere/pfo2-api-flask-sqlite>

GitHub Pages: <https://adrianojere.github.io/pfo2-api-flask-sqlite/>

1. Descripción del proyecto

API REST hecha con Flask que permite registrar usuarios con contraseñas hasheadas, iniciar sesión, y acceder a un endpoint de bienvenida al sistema de tareas. Los datos se guardan en SQLite. Incluye un cliente de consola que interactúa con la API usando la librería requests.

Método	Ruta	Descripción
POST	/registro	Crea un usuario nuevo. Recibe {"usuario", "contraseña"}
POST	/login	Verifica las credenciales y da acceso a las tareas
GET	/tareas	Devuelve un HTML de bienvenida

2. Cómo ejecutarlo

1. Instalar las dependencias: `pip install -r requirements.txt`
2. En una terminal, iniciar el servidor: `python servidor.py` (queda escuchando en `http://localhost:5000`)
3. En otra terminal, ejecutar el cliente: `python cliente.py`

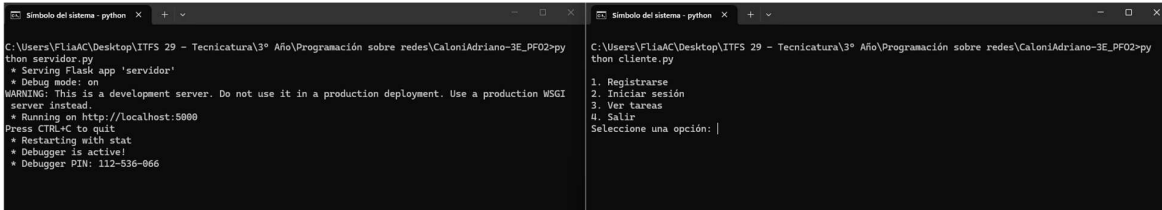
3. Resumen de pruebas

N°	Prueba	Resultado
1	Inicio del servidor y del cliente	Correcto
2	Registro con un usuario que ya existe	Correcto (error 409 controlado)
3	Login con contraseña incorrecta	Correcto (error 401 controlado)
4	Registro y login exitosos	Correcto
5	Acceso a /tareas tras iniciar sesión	Correcto
6	Verificación del hash en la base de datos	Correcto
7	Intento de ver tareas sin haber iniciado sesión	Correcto

4. Detalle de las pruebas

4.1 Inicio del servidor y del cliente

Se inicia el servidor Flask en localhost:5000 en una terminal, y el cliente de consola en otra, mostrando el menú de opciones.



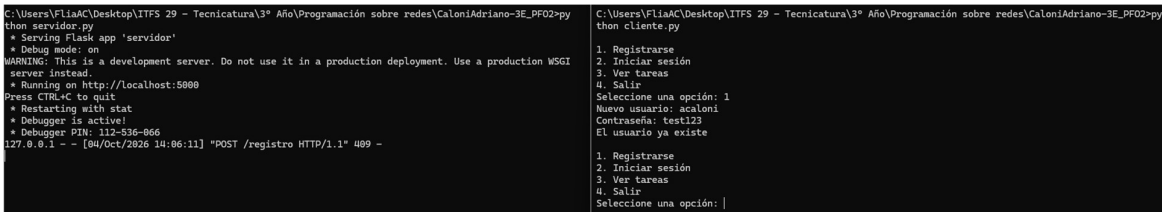
```
C:\Users\FliAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PF02>python servidor.py
* Serving Flask app 'servidor'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-536-866

C:\Users\FliAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PF02>python cliente.py
1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: |
```

Captura 1: servidor y cliente iniciados.

4.2 Registro con un usuario que ya existe

Se intenta registrar un usuario (acaloni) que ya estaba creado de una prueba anterior. El servidor responde con el error controlado "El usuario ya existe" (código HTTP 409), sin romperse.



```
C:\Users\FliAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PF02>python servidor.py
* Serving Flask app 'servidor'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-536-866
127.0.0.1 - - [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 -

C:\Users\FliAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PF02>python cliente.py
1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 1
Nuevo usuario: acaloni
Contraseña: test123
El usuario ya existe

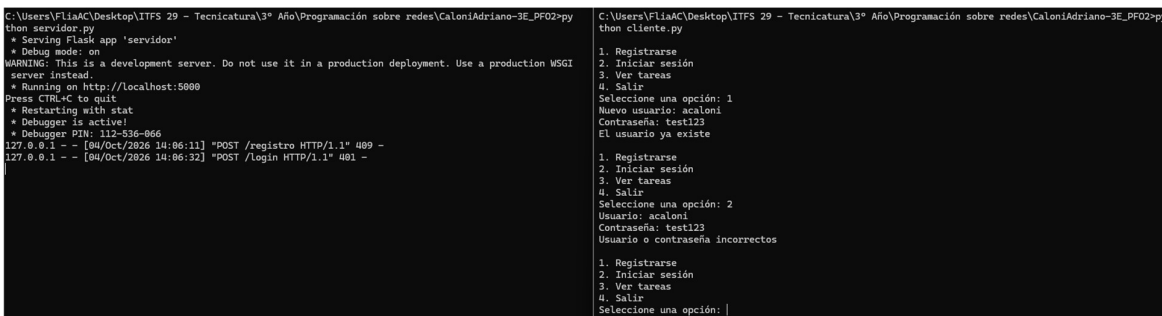
1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: |
```

Captura 2: error de registro por usuario duplicado.

Resultado: el servidor detecta el usuario duplicado y lo informa con un mensaje claro, en vez de guardar un registro repetido.

4.3 Login con contraseña incorrecta

Con el usuario acaloni ya registrado, se intenta iniciar sesión con una contraseña distinta a la original.



```
C:\Users\FliAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PF02>python servidor.py
* Serving Flask app 'servidor'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-536-866
127.0.0.1 - - [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:06:32] "POST /login HTTP/1.1" 401 -

C:\Users\FliAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PF02>python cliente.py
1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 1
Nuevo usuario: acaloni
Contraseña: test123
El usuario ya existe

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 2
Usuario: acaloni
Contraseña: test123
Usuario o contraseña incorrectos

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: |
```

Captura 3: login con contraseña incorrecta.

Resultado: el servidor responde "Usuario o contraseña incorrectos" (código HTTP 401) y no otorga acceso a /tareas.

4.4 Registro y login exitosos

Se registra un usuario nuevo (jcaloni) y se inicia sesión con esas mismas credenciales.

<pre>C:\Users\FliuAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PFD2>py thon servidor.py * Serving Flask app 'servidor' * Debug mode: on WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://localhost:5000 Press CTRL+C to quit * Restarting with stat * Debugger is active! * Debugger PIN: 112-536-066 127.0.0.1 -- [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 - 127.0.0.1 -- [04/Oct/2026 14:06:32] "POST /login HTTP/1.1" 401 - 127.0.0.1 -- [04/Oct/2026 14:07:07] "POST /login HTTP/1.1" 401 - 127.0.0.1 -- [04/Oct/2026 14:07:57] "POST /registro HTTP/1.1" 409 - 127.0.0.1 -- [04/Oct/2026 14:08:32] "POST /registro HTTP/1.1" 201 - 127.0.0.1 -- [04/Oct/2026 14:08:42] "POST /login HTTP/1.1" 200 -</pre>	<pre>C:\Users\FliuAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PFD2>py thon cliente.py 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 1 Nuevo usuario: jcaloni Contraseña: test123 El usuario ya existe 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 1 Nuevo usuario: jcaloni Contraseña: test1234 Usuario 'jcaloni' registrado correctamente 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 2 Usuario: jcaloni Contraseña: test1234 Bienvenido/a jcaloni. Ya podés acceder a /tareas 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: </pre>
--	---

Captura 4: registro y login exitosos.

Resultado: "Usuario 'jcaloni' registrado correctamente" y luego "Bienvenido/a jcaloni. Ya podés acceder a /tareas".

4.5 Acceso a /tareas tras iniciar sesión

Ya logueado como jcaloni, se elige la opción "Ver tareas" en el cliente.

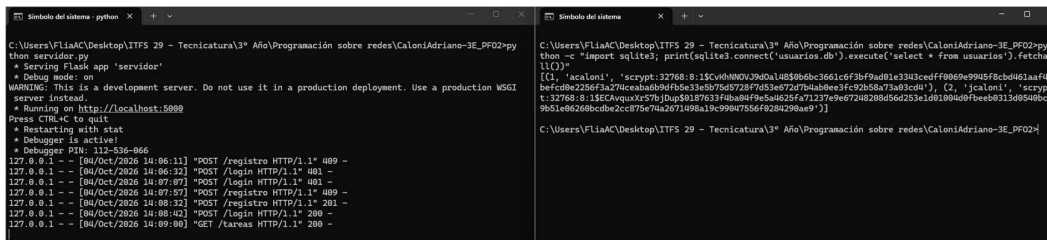
<pre>C:\Users\FliuAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PFD2>py thon servidor.py * Serving Flask app 'servidor' * Debug mode: on WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://localhost:5000 Press CTRL+C to quit * Restarting with stat * Debugger is active! * Debugger PIN: 112-536-066 127.0.0.1 -- [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 - 127.0.0.1 -- [04/Oct/2026 14:06:32] "POST /login HTTP/1.1" 401 - 127.0.0.1 -- [04/Oct/2026 14:07:07] "POST /login HTTP/1.1" 401 - 127.0.0.1 -- [04/Oct/2026 14:07:57] "POST /registro HTTP/1.1" 409 - 127.0.0.1 -- [04/Oct/2026 14:08:32] "POST /registro HTTP/1.1" 201 - 127.0.0.1 -- [04/Oct/2026 14:08:42] "POST /login HTTP/1.1" 200 - 127.0.0.1 -- [04/Oct/2026 14:09:00] "GET /tareas HTTP/1.1" 200 -</pre>	<pre>C:\Users\FliuAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PFD2>py thon cliente.py 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 1 Nuevo usuario: jcaloni Contraseña: test123 El usuario ya existe 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 1 Nuevo usuario: jcaloni Contraseña: test1234 Usuario 'jcaloni' registrado correctamente 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 2 Usuario: jcaloni Contraseña: test1234 Bienvenido/a jcaloni. Ya podés acceder a /tareas 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: 3 --- Respuesta del servidor (/tareas) --- <html> <head><title>Gestión de Tareas</title></head> <body style="font-family: Arial, sans-serif; text-align: center; margin-top: 60px;"> <h1>Bienvenido/a al Sistema de Gestión de Tareas</h1> <p>¡Inicia sesión en /login para gestionar tus tareas.</p> </body> </html> 1. Registrarse 2. Iniciar sesión 3. Ver tareas 4. Salir Seleccione una opción: </pre>
---	--

Captura 5: acceso a /tareas tras login.

Resultado: se recibe el HTML de bienvenida al sistema de gestión de tareas. En la terminal del servidor se ve el log GET /tareas HTTP/1.1" 200.

4.6 Verificación del hash en la base de datos

Para comprobar que la contraseña nunca se guarda en texto plano, se consultó directamente la tabla usuarios de usuarios.db con: `python -c "import sqlite3; print(sqlite3.connect('usuarios.db').execute('select * from usuarios').fetchall())"`



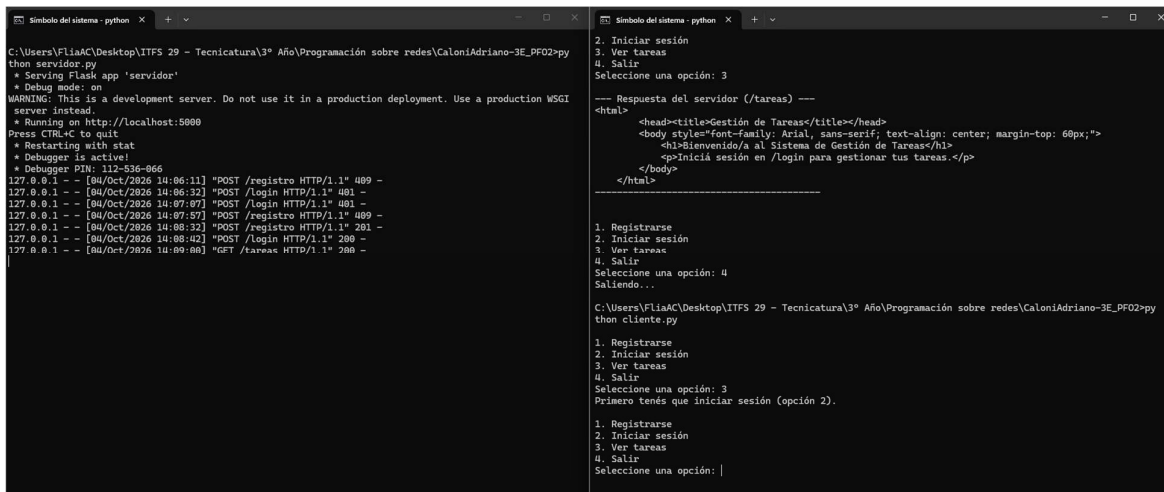
```
C:\Users\Fliac\Desktop\ITFS 29 - Tecnicatura3º Año\Programación sobre redes\CaloniAdriano-3E_PFO2.py
thor servidor.py
* Serving Flask app 'servidor'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-536-066
127.0.0.1 - - [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:06:32] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [04/Oct/2026 14:07:07] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [04/Oct/2026 14:07:57] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:08:32] "POST /registro HTTP/1.1" 201 -
127.0.0.1 - - [04/Oct/2026 14:08:42] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [04/Oct/2026 14:09:00] "GET /tarear HTTP/1.1" 200 -
```

Captura 6: consulta a la tabla usuarios mostrando los hashes.

Resultado: los dos usuarios registrados (acaloni y jcaloni) tienen cada uno un hash script distinto e irreversible en contraseña_hash, nunca la contraseña en texto plano.

4.7 Intento de ver tareas sin haber iniciado sesión

Al reiniciar el cliente y elegir directamente la opción "Ver tareas" sin pasar antes por el login, el cliente no deja continuar.



```
C:\Users\Fliac\Desktop\ITFS 29 - Tecnicatura3º Año\Programación sobre redes\CaloniAdriano-3E_PFO2.py
thor servidor.py
* Serving Flask app 'servidor'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-536-066
127.0.0.1 - - [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:06:32] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [04/Oct/2026 14:07:07] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [04/Oct/2026 14:07:57] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:08:32] "POST /registro HTTP/1.1" 201 -
127.0.0.1 - - [04/Oct/2026 14:08:42] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [04/Oct/2026 14:09:00] "GET /tarear HTTP/1.1" 200 -

1. Iniciar sesión
2. Ver tareas
3. Salir
Seleccione una opción: 3

--- Respuesta del servidor (/tarear) ---
<html>
<head><title>Gestión de Tareas</title></head>
<body style="font-family: Arial, sans-serif; text-align: center; margin-top: 60px;">
<h1>Bienvenido/a al Sistema de Gestión de Tareas</h1>
<p>Iniciá sesión en /login para gestionar tus tareas.</p>
</body>
</html>

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 4
Saliendo...

C:\Users\Fliac\Desktop\ITFS 29 - Tecnicatura3º Año\Programación sobre redes\CaloniAdriano-3E_PFO2.py
thor cliente.py

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 3
Primero tenés que iniciar sesión (opción 2).

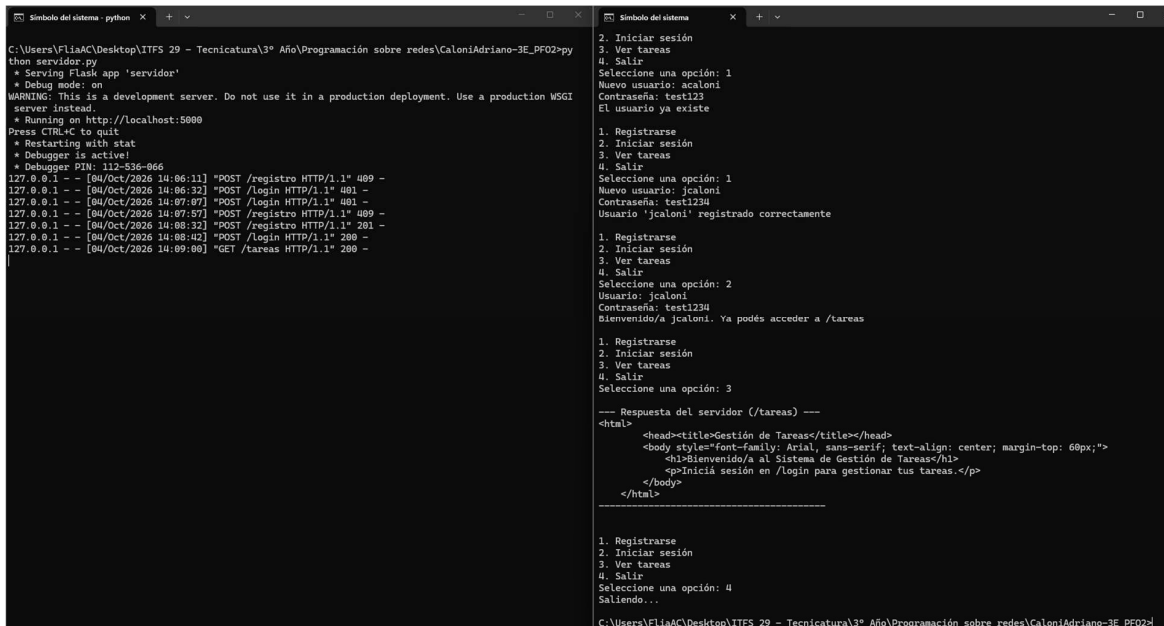
1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: |
```

Captura 7: intento de ver tareas sin iniciar sesión.

Resultado: el cliente responde "Primero tenés que iniciar sesión (opción 2)" y no llega a consultar /tarear.

4.8 Cierre del cliente

Al elegir la opción "Salir", el cliente termina de forma ordenada y devuelve el control a la terminal.



```
C:\Users\FLIAAC\Desktop\ITFS 29 - Tecnicatura\3º Año\Programación sobre redes\CaloniAdriano-3E_PFO2>python servidor.py
* Serving Flask app 'servidor'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active
* Debugger PIN: 112-536-066
127.0.0.1 - - [04/Oct/2026 14:06:11] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:06:32] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [04/Oct/2026 14:07:07] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [04/Oct/2026 14:07:57] "POST /registro HTTP/1.1" 409 -
127.0.0.1 - - [04/Oct/2026 14:08:32] "POST /registro HTTP/1.1" 201 -
127.0.0.1 - - [04/Oct/2026 14:08:42] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [04/Oct/2026 14:09:00] "GET /tarefas HTTP/1.1" 200 -

2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 1
Nuevo usuario: acaloni
Contraseña: test123
El usuario ya existe

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 1
Nuevo usuario: jcaloni
Contraseña: test1234
Usuario 'jcaloni' registrado correctamente

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 2
Usuario: jcaloni
Contraseña: test1234
Bienvenido/a jcaloni. Ya podés acceder a /tarefas

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 3

--- Respuesta del servidor (/tarefas) ---
<html>
  <head><title>Gestión de Tareas</title></head>
  <body style="font-family: Arial, sans-serif; text-align: center; margin-top: 60px;">
    <h1>Bienvenido/a al Sistema de Gestión de Tareas</h1>
    <p>Inicia sesión en /login para gestionar tus tareas.</p>
  </body>
</html>

1. Registrarse
2. Iniciar sesión
3. Ver tareas
4. Salir
Seleccione una opción: 4
Saliendo...
```

Captura 8: cierre del cliente.

5. Respuestas conceptuales

¿Por qué hashear contraseñas?

Porque si la base de datos se filtra o alguien accede a ella sin autorización, una contraseña en texto plano queda expuesta directamente. Un hash es el resultado de una función que transforma la contraseña en una cadena que no se puede revertir a la contraseña original. Al guardar el hash en vez del texto plano, aunque alguien acceda a la base de datos no puede leer las contraseñas reales, y para iniciar sesión alcanza con volver a hashear lo que escribe el usuario y compararlo con el hash guardado (check_password_hash), sin necesidad de descryptar nada.

Ventajas de usar SQLite en este proyecto

SQLite no necesita un proceso de servidor aparte: la base de datos es un solo archivo (usuarios.db) que se crea y se lee desde el propio código Python con el módulo sqlite3, que ya viene incluido con el lenguaje. Para un proyecto chico como este, eso evita instalar y configurar un motor de base de datos adicional (como MySQL o PostgreSQL), simplifica las pruebas locales (el archivo se puede borrar y se regenera solo) y alcanza de sobra en rendimiento para la cantidad de usuarios que maneja un trabajo práctico.

6. Nota sobre GitHub Pages

GitHub Pages solo puede servir contenido estático (HTML, CSS, JS): no puede ejecutar un servidor Flask en Python. Por eso, en docs/index.html se publicó una página con toda esta documentación (qué hace el proyecto, cómo ejecutarlo, las pruebas realizadas y las respuestas conceptuales) en lugar de la API en funcionamiento. Para probar la API hay que clonar el repositorio y ejecutarla localmente, como se explica en la sección 2.